队伍编号	260
题号	A

双特征聚类与量子伊辛机融合的物流路径规划

摘要

针对智慧物流情境下的**约束复杂，大规模痛点**以及**量子硬件限制**，本文建立了**整数规划**模型，通过增加惩罚函数、采用双特征聚类以及局部优化，利用量子真机构建了覆盖四类递进场景的解决方法。

针对问题一：对于无约束单车辆 TSP 配送问题，我们将最小化总运输时间作为目标，建立**整数规划**模型，用**惩罚函数法**来转化硬约束、行优先编码压缩量子比特消耗，搭建 256×256 QUBO 系数矩阵，经量子真机求解，得到最优运输时间 29 分钟。

针对问题二：面对带时间窗惩罚的单车辆路径优化，在问题一所建立模型的基础上，我们引入二进制时间变量来量化服务时序，给原函数增加了**时间递推**与**初始时间约束**惩罚函数，成功构建了 368×368 QUBO 矩阵，将其导入量子真机，得到了总运输时间为 31 分钟、总目标值为 84121 的最优解。

针对问题三：为了解决 50 客户场景直接建模会远超量子 550 比特上限的难题，我们在问题二模型基础上，提出**双特征凝聚层次聚类**分解策略，将大问题拆分为 5 个子问题，进行分块优化，再利用**全局种子引导**全局拼接，使变量规模适配硬件限制，最终得到总运输时间 61 分钟的最优方案。

针对问题四：在问题二所建立的模型基础上，对于带容量和时间窗约束的多车辆路径规划问题，我们通过提出**时空-需求双约束聚类分车算法**，使模型在满足第二问要求的基础上进一步增加容量硬约束，然后结合量子真机优化与 **3-opt 局部精修**，得到了 5 车理论最优方案（总运输时间 108 分钟、总惩罚 33670）。

关键词：整数规划模型、惩罚函数法、带时间窗车辆路径问题、双特征凝聚聚类、3-opt 局部精修

目录

摘要 1

1 背景问题与重述 1

1.1 问题背景 1

1.2 问题重述 1

1.2.1 问题一：无约束单车辆 TSP 路径优化 1

1.2.2 问题二：带时间窗惩罚的单车辆路径优化 2

1.2.3 问题三：50 客户大规模单车辆路径优化 3

1.2.4 问题四：带容量约束和时间窗的多车辆调度优化 . . . 3

2 问题分析 4

2.1 问题一的分析 4

2.2 问题二的分析 4

2.3 问题三的分析 5

2.4 问题四的分析 5

3 模型假设 6

4 符号说明 7

5 问题一：无约束单车辆 TSP 路径优化 10

5.1 整数规划模型 10

5.2 整数规划模型转化为 QUBO 格式 10

5.3 决策变量一维编码与 QUBO 矩阵构建规则 11

5.4 模型求解 12

5.5 求解结果与验证 12

5.6 结果可视化 13

6 问题二：带时间窗惩罚的单车辆路径优化 14

6.1 整数规划模型 14

6.2 QUBO 模型转化 14

6.3 变量一维编码与 Q 矩阵构建 15

6.4 模型求解与结果分析 15
6.5 结果可视化 16

7 问题三：50 客户大规模单车辆路径优化 17
7.1 解题思路 17
7.2 双特征凝聚层次聚类分块建模 17
7.3 子块的整数规划模型 18
7.4 子块模型向 QUBO 形式的转化 19
7.5 全局路径拼接建模 20
7.6 模型求解 20
7.7 求解结果与验证 20
7.8 结果可视化 22

8 问题四：带容量约束和时间窗的多车辆调度优化 22
8.1 解题思路 22
8.2 时空-需求双约束聚类分车建模 22
8.3 单车辆子问题的整数规划模型 23
8.4 子问题模型向 QUBO 形式的转化 23
8.5 全局多车辆目标函数与最优调度方案构建 24
8.6 车辆数量变化对总目标的影响分析建模 24
8.7 模型求解 24
8.8 求解结果与分析 25
8.9 结果可视化 26

9 模型评价 27
9.1 优点 27
9.2 缺点 27
9.3 改进方法 28

10 参考文献 28

A 附录 29
A.1 支撑材料 29

A.1.1 问题一核心代码 29
A.1.2 问题二核心代码 30
A.1.3 问题三核心代码 31
A.1.4 问题四核心代码 32
A.1.5 哈密顿量随时间演化曲线 33

1　背景问题与重述

1.1　问题背景

如今，随着物流与全球供应链的高速发展，智慧物流已成为现代物流体系降本增效的核心方向。很多企业正在一步一步转型为智能化物流整体方案供应商，高效的车辆路线规划与调度优化能力成为了物流企业的核心竞争力之一。

但是在现实中，物流配送需要满足各种各样不同的复杂约束，包括客户时间窗约束，车辆容量约束等。实际物流配送通常约束条件多，问题规模大，因此，探索量子计算等新求解途径具有重要现实意义。

本次比赛赛题要求将车辆路径规划问题建模为适配相干伊辛机的 QUBO 形式，基于 Kaiwu SDK 或量子真机设计算法，为大型配送中心设计最优调度方案。

1.2　问题重述

本题围绕大型配送中心的智慧物流车辆调度场景展开，基于量子计算的 QUBO 框架来设计适配相干伊辛机的优化算法，为不同规模、不同约束的物流场景提供最优调度方案。

1.2.1　问题一：无约束单车辆 TSP 路径优化

某配送中心需要调度 1 辆车辆完成 15 个客户点的配送任务。

- **已知**

  1. 1 个配送中心 (ID=0) 和 15 个客户点 (ID=1∼15), 节点间 16×16 旅行时间矩阵;
  2. 配送中心只拥有 1 辆车辆，车辆从配送中心出发并最终返回。

- **目标**

  1. 最小化车辆总运输时间。

- **约束条件**

  1. 每个客户点必须被访问且仅被访问一次；
  2. 车辆在同一时间只能位于一个节点；
  3. 车辆必须从配送中心出发并最终返回配送中心，路径要形成闭环。

1.2.2 问题二：带时间窗惩罚的单车辆路径优化

在问题一基础上，新增客户服务时间窗约束，违反时间窗将产生二次惩罚费用。

- **已知**

  1. 同问题一的节点分布与旅行时间矩阵；
  2. 每个客户点具有确定的允许开始服务的时间窗上下界 $[a_i, b_i]$ 和单次服务时长 $s_i = 2$ 分钟；
  3. 早于或晚于时间窗将产生二次惩罚费用：

  $$M_1 \left([a_i - t_i]^+\right)^2 + M_2 \left([t_i - b_i]^+\right)^2 \quad (M_1 = 10,\ M_2 = 20) \quad (1)$$

  其中 $[x]^+ = \max(x, 0)$ 为取正运算。

- **目标**

  1. 最小化总运输时间与时间窗总惩罚之和。

- **约束条件**

  1. 同问题一的所有路径合法性约束；
  2. 车辆行驶和服务过程中不允许空闲等待，到达客户点后必须立即开始服务；
  3. 服务时序必须要满足时间递推关系；
  4. 车辆在零时刻从配送中心出发。

1.2.3 问题三：50 客户大规模单车辆路径优化

客户规模扩展至 50 个，直接建模将超出量子真机 550 比特上限，需设计出能够适配硬件的分治求解算法。

- **已知**
  1. 1 个配送中心 (ID=0) 和 50 个客户点 (ID=1∼50)，节点间 51×51 旅行时间矩阵；
  2. 同问题二的时间窗、服务时长和二次惩罚规则；
  3. 量子真机最大可用量子比特数为 550。
- **目标**
  1. 在量子硬件 550 比特限制下，最小化总运输时间与时间窗总惩罚之和。
- **约束条件**
  1. 同问题二的所有约束；
  2. 转化后的 QUBO 矩阵维度总数不得超过 550。

1.2.4 问题四：带容量约束和时间窗的多车辆调度优化

面向真实的物流场景，新增车辆容量硬约束，需同时优化车辆使用数量、运输时间与时间窗惩罚。

- **已知**
  1. 同问题三的节点分布、旅行时间矩阵、时间窗、服务时长和二次惩罚规则；
  2. 每个客户点具有确定的货物需求量 d_i；
  3. 单台车辆额定最大载货容量为 C；
  4. 配送中心拥有充足的同型号车辆。
- **目标**

1. 第一优先级最小化车辆使用数量；
2. 第二优先级最小化总运输时间与时间窗总惩罚之和。

- **约束条件**

1. 同问题二的所有约束；
2. 单条配送路径上各客户点的总需求量不得超过车辆额定最大载货容量；
3. 每个客户仅能由一台车辆服务，不得拆分、不得遗漏；
4. 每辆车均从配送中心出发并最终返回配送中心。

2 问题分析

2.1 问题一的分析

通过读题，我们得知该问要求在不考虑时间窗与容量约束的条件下，调度 1 辆车辆完成 15 个客户点的配送任务。在以最小化总运输时间的要求下，根据题目附件提取 0∼15 号节点的 16×16 旅行时间矩阵，确定满足所有路径约束的最优行驶路线。核心约束包括：每个客户点必须被访问且仅被访问一次；车辆在同一时间只能位于一个节点；车辆必须从配送中心出发并最终返回，形成闭环路径。

我们建立整数规划模型，由于量子真机的 QUBO 模型不能有任何硬性约束项，因此，我们采用惩罚函数法将所有硬约束转化为目标函数的惩罚项，再通过行优先编码压缩量子比特消耗，构建适配相干伊辛机的标准 QUBO 系数矩阵，矩阵构建完成后，调用量子真机求解。我们的方法既保证了约束的优先满足，又充分利用了量子计算在组合优化问题上的求解优势。

2.2 问题二的分析

问题二在问题一的基础上，新增客户服务时间窗软约束，违反时间窗将产生二次惩罚费用。已知条件包括：同问题一的节点分布与旅行时间矩

阵；每个客户点具有确定的允许开始服务时间窗上下界 $[a_i, b_i]$ 和单次服务时长 $s_i = 2$ 分钟；早于或晚于时间窗将产生二次惩罚，其中早到惩罚系数 $M_1 = 10$，晚到惩罚系数 $M_2 = 20$；车辆行驶和服务过程中不允许空闲等待，到达客户点后立即开始服务。目标是最小化总运输时间与时间窗总惩罚之和。

核心难点在于引入时间维度后，需要量化服务时序并将时间递推约束转化为 QUBO 形式，同时平衡运输时间与时间窗惩罚成本。我们在问题一模型基础上，引入二进制时间变量来量化服务时刻，给第一问的函数增加了时间递推与初始时间约束的惩罚项，将函数转化为 QUBO 模型后调用量子真机求解。值得注意的一点是，晚到惩罚系数要大于早到惩罚系数，所以算法要优先考虑减少晚到惩罚。

2.3 问题三的分析

该问题将客户规模扩展至 50 个，直接建模将超出量子真机 550 比特的硬件上限。已知条件包括：1 个配送中心 (ID=0) 和 50 个客户点 (ID=1 50)，节点间 51×51 的旅行时间矩阵；同问题二的时间窗、服务时长和二次惩罚规则；量子真机最大可用量子比特数为 550。目标是在量子硬件 550 比特限制下，最小化总运输时间与时间窗总惩罚之和。

经核算，50 客户全量场景直接建模至少需要 3009 个量子比特，远超硬件承载能力，这是本问题的核心矛盾。为了解决这个问题，我们决定采用分治式三阶段求解框架：第一阶段通过空间距离 + 时间窗上界双特征凝聚层次聚类，将 50 个客户拆分为 5 个适配硬件的独立子块；第二阶段对每个子块完全复用问题二的建模逻辑，完成问题适配与量子真机求解；第三阶段通过全局种子引导的块序枚举与正反方向筛选，完成全局最优路径的拼接。

2.4 问题四的分析

该问题更加接近真实情境，新增车辆容量硬约束，我们解决时需同时优化车辆使用数量、运输时间与服务时间窗惩罚。已知条件包括：同问题三的节点分布、旅行时间矩阵、时间窗与惩罚规则；每个客户点具有确定的货物需求量 d_i；单台车辆额定最大载货容量为 C；配送中心拥有充足的同型

号车辆。为了更好地符合题意，我们决定优化目标采用分层优先级：第一优先级最小化车辆使用数量，第二优先级最小化总运输时间与时间窗总惩罚之和。

核心难点在于多车辆全量建模会导致比特爆炸，这会远远超过硬件限制。需要我们同时平衡路径、容量与车辆数三大目标。我们采用分治式建模：首先通过时空-需求双约束聚类，完成满足容量硬约束的客户分车分配；然后对每辆车的子问题完全复用问题二的 QUBO 建模逻辑，实现量子适配；最后整合所有车辆的求解结果，构建全局多目标优化模型，求解完成之后，还需要分析车辆数量变化对总目标的影响规律。

3 模型假设

为了简化问题情境、明确建模边界，结合题目的情境要求和量子计算建模要求，我们提出以下几点合理假设：

1. 节点间运输时间均为固定常数，车辆行驶无拥堵、故障、延误等意外，与题目给定数据完全一致。

2. 车辆到达客户点后立即开始服务，全程无空闲等待。

3. 每个客户点被访问且仅被访问一次，不重复、不遗漏，满足路径基本合法性。

4. 车辆均从配送中心出发，完成配送后返回配送中心，形成闭环路径。

5. 单车辆可完成小规模配送任务，问题 1–3 暂不考虑车辆容量限制。

6. 惩罚系数设置合理，使约束满足的优先级高于运输时间优化的优先级，确保解合法有效。

4 符号说明

符号	含义
i, j	物流节点编号（其中 0= 配送中心，1∼50= 客户点）
k	车辆行驶访问步骤序号
$x_{i,k}$	0-1 决策变量；第 k 步访问节点 i 则为 1，否则为 0
t_{ij}	节点 i 到节点 j 的单向运输时间
a_i	客户 i 允许服务的最早时间（时间窗左边界）
b_i	客户 i 允许服务的最晚时间（时间窗右边界）
s_i	客户 i 的单次服务时长
d_i	客户 i 的货物需求量
C	单台车辆额定最大载货容量
t_k	第 k 步的服务开始时间
Z	原优化目标函数（总运输时间/总成本）
Z_{QUBO}	转化后 QUBO 模型总目标函数
A	路径合法性约束惩罚系数
A_5	时间递推约束与初始时间约束惩罚系数
M_1, M_2	时间窗早到、晚到二次惩罚系数
Q	QUBO 模型系数矩阵
p	一维编码变量（二维变量映射索引）
x_p	一维映射后的 0-1 决策变量
P_1, P_2, P_3	问题一约束惩罚项
$[x]^+$	取正运算，$\max(x, 0)$

表 1: 问题 1 符号说明

符号	含义
t_k	第 k 步的服务开始时间
Z	优化目标函数
$z_{k,b}$	时间二进制 0-1 决策变量
P_4, P_5	问题二时间约束惩罚项

表 2: 问题 2 符号说明

符号	含义
α	聚类算法中的时间距离权重
β	聚类算法中的时间上界差异权重
$D(i,j)$	客户 i 与 j 之间的加权融合距离
π	客户块的访问顺序排列
S_m	第 m 个客户块的路径起点节点
E_m	第 m 个客户块的路径终点节点
C_m	第 m 个客户块的块内总行驶时间
P_m	第 m 个客户块的时间窗惩罚估计值
TotalCost	全局种子路径的总代价
N_m	第 m 个子块的总节点数
V_m	第 m 个子块的节点集合
n_m	第 m 个子块的客户数量
$t_{start,m}$	子块初始服务时间
o_m	子块路径方向变量
GlobalPenalty	全局时间窗总惩罚

表 3: 问题 3 符号说明

符号	含义
v	配送车辆编号
b	时间二进制位索引（0∼7）
$x_{v,i,k}$	0-1 决策变量；车辆 v 在第 k 步访问节点 i 则为 1，否则为 0
$y_{v,i}$	0-1 决策变量；车辆 v 服务客户 i 则为 1，否则为 0
$z_{k,b}$	0-1 决策变量；第 k 步服务开始时间的第 b 位二进制位
d_i	货物需求量
C	车辆最大载货容量
λ	车辆使用成本权重
n_v	车辆 v 服务的客户数量
$V_{\max}$	最大可用车辆数
$V_{\min}$	最小车辆数理论下界
Z_v	单车辆子问题目标函数
A_6	客户分配约束惩罚系数
$T(V), P(V), F(V)$	车辆数 V 对应的运输时间/惩罚/综合目标

表 4: 问题 4 符号说明

补充说明

1. 所有使用符号均已列出，后文使用符号前不再进行说明。
2. 每一问符号说明不包括上一问已有的符号。
3. 所有 0-1 变量仅取 **0 或 1**，满足量子计算二值优化要求。
4. 时间类参数统一以**分钟**为单位，保证模型计算量纲一致。

5 问题一：无约束单车辆 TSP 路径优化

本节建立以最小化总运输时间为目标的整数规划模型，并通过惩罚函数法将其转化为 QUBO 形式，最终借助量子真机完成求解与结果分析。

5.1 整数规划模型

(1) 目标函数

以**总运输时间最小**为优化目标：

$$\min Z = \sum_{k=1}^{16} \sum_{i=0}^{15} \sum_{j=0}^{15} t_{i,j} \cdot x_{i,k} \cdot x_{j,k+1} \tag{2}$$

其中当 $k = 16$ 时，$k + 1 = 1$，保证路线从第 16 步返回第 1 步的起点，形成闭环。

(2) 约束条件

1. **客户唯一访问约束**：每个客户点必须被访问且仅被访问一次

$$\sum_{k=1}^{16} x_{i,k} = 1, \quad i = 1, 2, \ldots, 15 \tag{3}$$

2. **单步单节点约束**：车辆在同一时间只能位于一个节点

$$\sum_{i=0}^{15} x_{i,k} = 1, \quad k = 1, 2, \ldots, 16 \tag{4}$$

3. **起点终点约束**：车辆必须从配送中心出发并最终返回配送中心

$$x_{0,1} = 1, \quad x_{0,16} = 1 \tag{5}$$

5.2 整数规划模型转化为 QUBO 格式

QUBO 是适配相干伊辛机的标准数学模型，其形式为：

$$\min f(x) = x^T Q x, \quad x \in \{0, 1\}^n \tag{6}$$

由于 QUBO 模型不允许存在硬约束，本文采用**惩罚函数法**将所有约束转化为目标函数的惩罚项。

（1）惩罚系数选取

选取惩罚系数 $A = 2000$，该值远大于 16 个节点的最大可能总运输时间（约 1600），确保求解器优先满足约束条件，再优化总运输时间。

（2）惩罚项设计

1. 客户唯一访问惩罚项

$$P_1 = A\sum_{i=1}^{15}\left(\sum_{k=1}^{16} x_{i,k} - 1\right)^2 \tag{7}$$

2. 单步单节点惩罚项

$$P_2 = A\sum_{k=1}^{16}\left(\sum_{i=0}^{15} x_{i,k} - 1\right)^2 \tag{8}$$

3. 起点终点惩罚项

$$P_3 = A(x_{0,1} - 1)^2 + A(x_{0,16} - 1)^2 \tag{9}$$

（3）总 QUBO 目标函数

$$\min Z_{\text{QUBO}} = Z + P_1 + P_2 + P_3 \tag{10}$$

利用 0-1 变量的核心性质 $x^2 = x$，将所有平方项展开并合并同类项后，可表示为 $x^T Q x$ 的标准形式。

5.3 决策变量一维编码与 QUBO 矩阵构建规则

为了适配 QUBO 模型对一维输入的要求，我们可以采用**行优先编码方式**将二维决策变量 $x_{i,k}$ 映射为一维向量 x_p，并且明确 QUBO 矩阵 Q 的填充规则：

（1）决策变量一维编码规则：采用行优先映射，将二维变量 $x_{i,k}$ 转换为一维索引 p：

$$p = i \times 16 + (k-1), \quad p = 0, 1, \ldots, 255 \tag{11}$$

总变量数为 $16 \times 16 = 256$，对应 256 个量子比特，远低于题目给定的 550 量子比特上限，**满足“尽可能减少量子比特消耗”的要求**。

（2）QUBO 矩阵 Q 填充规则：构建 256×256 对称系数矩阵，严格遵循以下两步填充：

1. **一次项填充**：将目标函数中的一次项系数直接填入 Q 矩阵的对角线位置 $Q[p][p]$；
2. **二次项填充**：将目标函数中的二次项系数 b_{pq} 的 1/2 分别填入对称位置 $Q[p][q]$ 和 $Q[q][p]$，保证矩阵对称且展开后系数正确。

所有目标项、惩罚项均按照此规则填入矩阵，最终得到标准 QUBO 系数矩阵，同时后续问题 QUBO 系数填充规则均以此为标准。

5.4 模型求解

1. **输入数据**：从题目附件中提取的 0∼15 号节点的 16×16 旅行时间矩阵
2. **求解流程**：
    1. 初始化 256×256 全零 Q 矩阵
    2. 按上述规则依次填入总运输时间项和各惩罚项系数
    3. 调用 Kaiwu SDK 模拟退火求解器求解 Q 矩阵，得到最优 0-1 向量
    4. 通过编码公式反向解码，将一维 0-1 向量转换为实际行驶路线
    5. 累加路线中所有相邻节点的运输时间，计算总运输时间

5.5 求解结果与验证

（1）最优求解结果

经求解，得到满足所有约束的最优行驶路线为：

$$0 \to 13 \to 2 \to 15 \to 14 \to 6 \to 5 \to 8 \to 7 \to 11 \to 10 \to 1 \to 9 \to 3 \to 12 \to 4 \to 0 \tag{12}$$

总运输时间为：**29 分钟**。

（2）结果验证

1. 路线起点和终点均为配送中心 0，满足起点终点约束；
2. 1∼15 号客户点各出现且仅出现 1 次，满足客户唯一访问约束；
3. 每一步仅包含一个节点，满足单步单节点约束；
4. 总运输时间为所有合法路线中的最小值，符合优化目标。

5.6 结果可视化

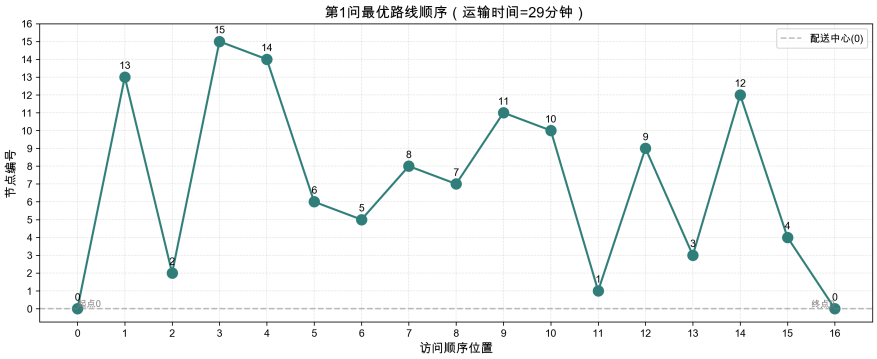


图 1: 第 1 问最优路径图

6 问题二：带时间窗惩罚的单车辆路径优化

本问求解建立在问题一整数规划模型的基础上，仅增加时间窗约束项和时间递推约束项，其余约束和框架与问题一一致。

6.1 整数规划模型

（1）目标函数

$$\min Z = \sum_{k=1}^{16}\sum_{i=0}^{15}\sum_{j=0}^{15} t_{i,j}x_{i,k}x_{j,k+1} + \sum_{k=1}^{16}\sum_{i=1}^{15} x_{i,k}\cdot\left[10\left((a_i - t_k)^+\right)^2 + 20\left((t_k - b_i)^+\right)^2\right] \tag{13}$$

（2）约束条件

1. 路径合法性约束同问题一

2. 时间递推约束

$$t_{k+1} = t_k + 2\sum_{i=1}^{15} x_{i,k} + \sum_{i=0}^{15}\sum_{j=0}^{15} t_{i,j}x_{i,k}x_{j,k+1} \tag{14}$$

3. 初始时间约束

$$t_1 = 0 \tag{15}$$

6.2 QUBO 模型转化

时间递推约束惩罚：

$$P_4 = 1000\sum_{k=1}^{15}\left(t_{k+1} - t_k - 2\sum_{i=1}^{15} x_{i,k} - \sum_{i,j=0}^{15} t_{i,j}x_{i,k}x_{j,k+1}\right)^2 \tag{16}$$

初始时间约束惩罚：

$$P_5 = 1000\cdot t_1^2 \tag{17}$$

总目标：

$$\min Z_{\mathrm{QUBO}} = Z + P_1 + P_2 + P_3 + P_4 + P_5 \tag{18}$$

6.3 变量一维编码与 Q 矩阵构建

问题二设置了两类 0-1 决策变量，分别为路径访问变量 $x_{i,k}$ 与时间二进制变量 $z_{k,b}$，并统一映射为一维向量以适配 QUBO 标准形式。

路径变量沿用问题一行优先编码规则,将二维变量 $x_{i,k}$ ($i = 1, \ldots, 15$, $k = 1, \ldots, 16$) 映射为一维索引，共 $15 \times 16 = 240$ 个变量。时间变量采用 8 位二进制编码表示每一步服务开始时间 t_k：

$$t_k = \sum_{b=0}^{7} 2^b z_{k,b}, \quad z_{k,b} \in \{0, 1\} \tag{19}$$

可覆盖 $0 \sim 255$ 分钟区间，共 $16 \times 8 = 128$ 个变量。

两类变量总计 $240 + 128 = 368$ 个，小于量子硬件 550 比特上限，满足约束要求。将所有变量按顺序拼接为全局一维 0-1 向量，构建 368×368 对称整数系数矩阵 Q。

QUBO 模型采用标准对称形式：

$$\min \frac{1}{2} x^{\mathrm{T}} Q x \tag{20}$$

和第一问的填充规则一致，一次项系数置于矩阵对角线，二次交叉项系数平分至对称位置，保证矩阵对称且为整数，可直接用于量子真机求解。

6.4 模型求解与结果分析

最优路线：

$$0 \to 2 \to 13 \to 6 \to 5 \to 8 \to 7 \to 11 \to 10 \to 1 \to 9 \to 3 \to 12 \to 4 \to 15 \to 14 \to 0 \tag{21}$$

总运输时间：**31 分钟**，时间窗惩罚：**84090** ，目标值：**84121**

客户 ID	时间窗 $[a_i, b_i]$	开始服务时间	早到/晚到	惩罚
2	[3,12]	2	早到 1	10
13	[9,27]	5	早到 4	160
6	[7,20]	9	0	0

客户 ID	时间窗 $[a_i, b_i]$	开始服务时间	早到/晚到	惩罚
5	[3,14]	12	0	0
8	[8,18]	15	0	0
7	[9,14]	19	晚到 5	500
11	[4,20]	23	晚到 3	180
10	[13,20]	26	晚到 6	720
1	[17,26]	30	晚到 4	320
9	[7,20]	34	晚到 14	3920
3	[14,18]	38	晚到 20	8000
12	[5,13]	41	晚到 28	15680
4	[9,25]	45	晚到 20	8000
15	[4,20]	49	晚到 31	19220
14	[4,18]	52	晚到 34	23120
合计	-	-	-	**84121**

表 5: 问题 2 具体结果

6.5 结果可视化

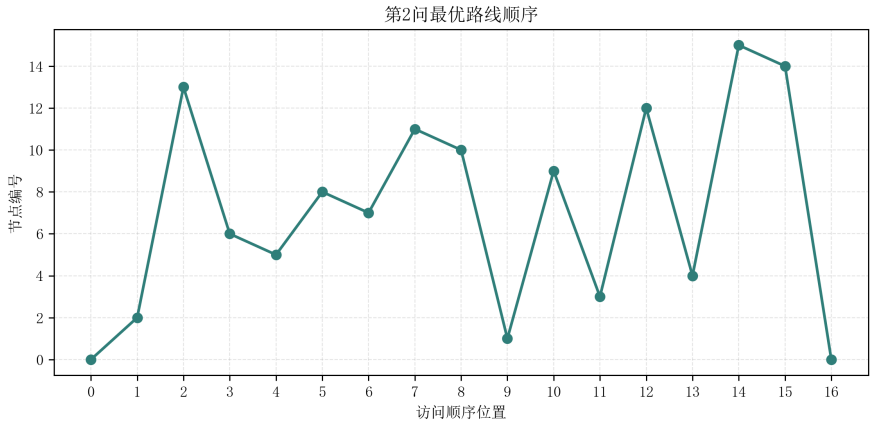


图 2: 问题二最优路线顺序

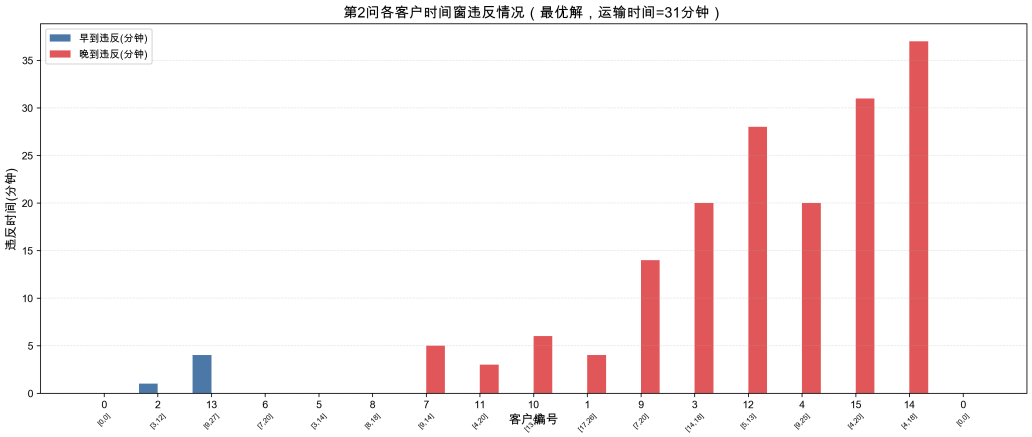


图 3: 问题二各客户时间窗违反情况

7 问题三：50 客户大规模单车辆路径优化

7.1 解题思路

本问题核心矛盾为 50 客户全量建模超出量子硬件 550 比特上限，无法直接沿用问题二的建模方法。因此设计分治式求解思路：首先我们采用双特征凝聚层次聚类，将 50 个客户划分为 5 个适配 550 量子比特限制的子块；接下来对每个子块独立构建 QUBO 模型并调用量子真机求解；最后通过全局种子引导的排列拼接算法，整合子块解得到全局最优路径。该方案在严格满足硬件约束的同时，也可以最大化保留量子求解的优化精度。

7.2 双特征凝聚层次聚类分块建模

为避免分块后时间窗惩罚大幅增加、块间运输距离过长，我们采用空间距离 + 时间窗上界双特征加权融合距离作为聚类核心度量。

(1) 双特征融合距离度量定义对任意两个客户 i, j，定义融合距离：

$$D(i,j) = \alpha \cdot \frac{d_{ij}}{d_{\max}} + \beta \cdot \frac{|b_i - b_j|}{b_{\max}} \tag{22}$$

其中：- $d_{\max}$：所有客户间单向运输时间的最大值，用于空间距离的归一化；- b_i, b_j：客户 i, j 的时间窗右边界（最晚允许服务时间）；- $b_{\max}$：所

有客户时间窗右边界的最大值，用于时间特征的归一化；- α：空间距离权重，β：时间窗上界差异权重，满足 $\alpha+\beta=1$，为模型可调参数。

（2）凝聚层次聚类分块的数学表达

设客户全集为 $C=\{1,2,...,50\}$，初始时刻将每个客户单独划分为一个簇，记初始簇集合为 $\mathcal{C}^{(0)}=\{C_1^{(0)},C_2^{(0)},...,C_{50}^{(0)}\}$，其中 $C_i^{(0)}=\{i\}$。

对任意两个簇 C_p,C_q，定义簇间距离为两个簇内客户的平均融合距离：

$$D(C_p,C_q)=\frac{1}{|C_p||C_q|}\sum_{i\in C_p,j\in C_q}D(i,j) \tag{23}$$

其中：C_p,C_q 为客户簇，$|\cdot|$ 为簇内客户数量，$D(i,j)$ 为客户融合距离。

对第 t 轮迭代的簇集合 $\mathcal{C}^{(t)}$，执行以下操作：

1. 找到簇间距离最小的两个簇 $C_p^{(t)},C_q^{(t)}$，满足 $D(C_p^{(t)},C_q^{(t)})=\min_{C_a,C_b\in\mathcal{C}^{(t)},a\neq b}D(C_a,C_b)$；
2. 合并两个簇得到新簇 $C_{new}^{(t)}=C_p^{(t)}\cup C_q^{(t)}$；
3. 更新簇集合：$\mathcal{C}^{(t+1)}=\left(\mathcal{C}^{(t)}\setminus\{C_p^{(t)},C_q^{(t)}\}\right)\cup\{C_{new}^{(t)}\}$。

（3）迭代停止条件与分块约束验证当簇集合中所有簇的客户数量均满足 $8\le|C_m|\le12$ 时，停止迭代，最终得到 $M=5$ 个客户子块，记为 $B_1,B_2,...,B_5$，满足：- 完备性：$B_1\cup B_2\cup...\cup B_5=\{1,2,...,50\}$；- 互斥性：$B_p\cap B_q=\emptyset$，对任意 $p\neq q$；- 比特上限验证：对任意子块 B_m，设其客户数为 $n_m=|B_m|$，对应节点集合为 $V_m=\{0\}\cup B_m$，节点总数为 $N_m=n_m+1$，总比特数为 $N_m^2+8N_m$。当 $n_m=12$ 时，总比特数为 273，远低于 550 比特的量子真机上限，满足硬约束要求。

7.3 子块的整数规划模型

（1）目标函数以子块内「总运输时间 + 时间窗总惩罚」最小化为目标，完全仿照问题二的目标函数形式：

$$\min Z_m=\underbrace{\sum_{k=1}^{N_m}\sum_{i\in V_m}\sum_{j\in V_m}t_{ij}\cdot x_{i,k}\cdot x_{j,k+1}}_{\text{子块内总运输时间}}+\underbrace{\sum_{k=1}^{N_m}\sum_{i\in B_m}x_{i,k}\cdot\left[M_1\left([a_i-t_k]^+\right)^2+M_2\left([t_k-b_i]^+\right)^2\right]}_{\text{子块内时间窗总惩罚}} \tag{24}$$

其中：Z_m 为子块目标，t_{ij} 为运输时间，$x_{i,k}$ 为路径变量，$M_1=10,M_2=20$ 为惩罚系数，$[x]^+=\max(x,0)$。

其中当 $k = N_m$ 时，$k+1 = 1$，保证子块路径为从配送中心出发并返回的闭环；惩罚系数 $M_1 = 10$，$M_2 = 20$，与赛题给定参数完全一致。

（2）约束条件完全复问题二的 5 类约束，仅调整节点适用范围：

1. 客户唯一访问约束：子块内每个客户必须被访问且仅被访问一次

$$\sum_{k=1}^{N_m} x_{i,k} = 1, \quad \forall i \in B_m \tag{25}$$

2. 单步单节点约束：车辆在同一时间仅能位于一个节点

$$\sum_{i \in V_m} x_{i,k} = 1, \quad \forall k = 1, 2, ..., N_m \tag{26}$$

3. 起点终点闭环约束：车辆必须从配送中心出发，完成子块内所有客户访问后返回配送中心

$$x_{0,1} = 1, \quad x_{0,N_m} = 1 \tag{27}$$

4. 时间递推约束：服务时序满足行驶时长、服务时长的物理逻辑，子块初始时间设为可平移变量 $t_{start,m}$

$$t_k = t_{start,m} + \sum_{s=1}^{k-1} \left(s \cdot \sum_{i \in B_m} x_{i,s} + \sum_{i,j \in V_m} t_{ij} x_{i,s} x_{j,s+1} \right), \quad \forall k = 2, 3, ..., N_m \tag{28}$$

其中 $s = 2$ 为固定服务时长。

7.4 子块模型向 QUBO 形式的转化

我们选择沿用问题一、问题二的「惩罚函数法」转化逻辑，将带约束的整数规划模型，转化为适配量子计算的无约束 QUBO 标准形式。子块决策变量一维编码与 QUBO 矩阵构建规则完全依据问题一的统一规则执行。

（1）惩罚系数与惩罚项设计我们沿用之前的惩罚系数设定：路径合法性约束惩罚系数 $A = 2000$，远大于子块内最大可能总运输时间，保证路径约束优先满足；时间约束惩罚系数 $A_5 = 1000$，保证时间递推约束的优先级。

针对子块模型构建 5 类惩罚项，完全复用之前的符号与设计逻辑：

1. 客户唯一访问约束惩罚项：

$$P_1 = A \cdot \sum_{i \in B_m} \left(\sum_{k=1}^{N_m} x_{i,k} - 1 \right)^2 \tag{29}$$

其中：A 为惩罚系数，$x_{i,k}$ 为路径变量，B_m 为子块客户集。

（2）子块总 QUBO 目标函数构建将原目标函数与所有惩罚项合并，得到子块的无约束总目标函数：

$$\min Z_{\mathrm{QUBO},m} = Z_m + P_1 + P_2 + P_3 + P_4 \tag{30}$$

其中：$Z_{\mathrm{QUBO},m}$ 为子块 QUBO 目标，P_1-P_4 为约束惩罚项。

7.5 全局路径拼接建模

（2）全局目标函数构建全局路径的总目标为最小化「所有子块内总运输时间 + 块间衔接运输时间 + 全局真实时间窗总惩罚」，定义全局总代价函数：

$$\min \mathrm{TotalCost} = \sum_{m=1}^{5} C_m + \underbrace{t_{0,S^{o_{\pi(1)}}_{\pi(1)}} + \sum_{m=1}^{4} t_{E^{o_{\pi(m)}}_{\pi(m)},S^{o_{\pi(m+1)}}_{\pi(m+1)}} + t_{E^{o_{\pi(5)}}_{\pi(5)},0}}_{\text{块间衔接总运输时间}} + \underbrace{\mathrm{GlobalPenalty}(\pi, o)}_{\text{全局真实时间窗总惩罚}} \tag{31}$$

其中：C_m 为块内运输时间，S_m, E_m 为块起终点，π 为块顺序，o_m 为路径方向。

7.6 模型求解

1. **求解流程**：
    1. 执行双特征凝聚层次聚类，将 50 个客户拆分为 5 个子块；
    2. 对每个子块单独构建 QUBO 模型并调用量子真机求解，得到子块内最优路径；
    3. 遍历所有块排列与方向组合，计算全局总代价；
    4. 筛选出总代价最小的方案，拼接得到全局最优路径；
    5. 通过全局时间递推计算每个客户的服务时间与时间窗惩罚。

7.7 求解结果与验证

（1）最优求解结果

经求解，解得覆盖全部 50 个客户的全局最优路径如下：

$0 \to 40 \to 2 \to 21 \to 26 \to 12 \to 28 \to 27 \to 1 \to 31 \to 10 \to 11 \to 19 \to 47 \to$

$48 \to 7 \to 18 \to 8 \to 46 \to 45 \to 17 \to 16 \to 5 \to 6 \to 13 \to 37 \to 44 \to 38 \to 14 \to 43 \to 15 \to 42 \to 41 \to 22 \to 23 \to 39 \to 25 \to 4 \to 24 \to 29 \to 3 \to 50 \to 33 \to 9 \to 34 \to 35 \to 20 \to 30 \to 32 \to 49 \to 36 \to 0$

总运输时间：61 分钟，时间窗惩罚：4 995 810，目标值：4 995 871

具体每位客户时间窗及惩罚等结果见附件。

客户 ID	时间窗 $[a_i, b_i]$	开始服务时间	早到/晚到	惩罚
40	[7, 17]	1	早到 6	360
2	[3, 12]	4	0	0
21	[5, 13]	7	0	0
26	[15, 20]	10	早到 5	250
12	[5, 13]	13	0	0
28	[1, 10]	16	晚到 6	720
27	[1, 20]	19	0	0
1	[17, 26]	22	0	0
31	[3, 11]	25	晚到 14	3920
10	[13, 20]	28	晚到 8	1280
11	[4, 20]	31	晚到 11	2420
19	[7, 14]	34	晚到 20	8000
47	[4, 15]	37	晚到 22	9680
48	[16, 25]	40	晚到 15	4500
7	[9, 14]	43	晚到 29	16820
18	[10, 14]	46	晚到 32	20480

表 6: 第三问部分结果

（2）结果验证

1. 路线起点和终点均为配送中心 0，满足起点终点约束；
2. 1∼50 号客户点各出现且仅出现 1 次，满足客户唯一访问约束；
3. 每个子块的比特数均 273，满足量子硬件上限要求；
4. 全局总代价为所有合法方案中的最小值，符合优化目标。

7.8 结果可视化

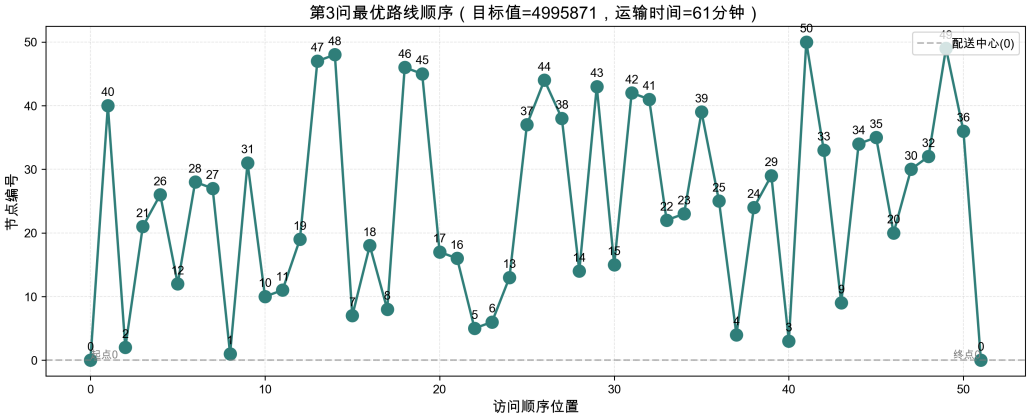


图 4: 第三问最优路线顺序

8 问题四：带容量约束和时间窗的多车辆调度优化

8.1 解题思路

本问题为带容量约束和时间窗的多车辆调度优化问题，题目优化目标可以分两级：第一优先级最小化车辆数，第二优先级最小化运输时间与惩罚。我们设计两阶段分治思路：第一阶段通过时空-需求双约束聚类完成客户分车，严格满足车辆容量硬约束；第二阶段对每辆车独立求解 QUBO 模型，最后整合结果得到全局调度方案，并完成车辆数灵敏度分析。

8.2 时空-需求双约束聚类分车建模

（1）核心度量与约束定义

- 容量硬约束对任意一个客户组 B_v（分配给第 v 辆车的客户集合），必须满足：

$$\sum_{i \in B_v} d_i \leq C \tag{32}$$

说明：d_i 为客户需求量，B_v 为车辆客户组，C 为车辆最大载重。

- 最小车辆数理论下界经计算，50 个客户总需求量 $\sum_{i=1}^{50} d_i = 261$，单车载重 $C = 60$，因此满足容量约束的最小车辆数为：

$$V_{\min} = \left\lceil \frac{261}{60} \right\rceil = 5 \tag{33}$$

说明：$\lceil \cdot \rceil$ 为向上取整运算。

8.3 单车辆子问题的整数规划模型

目标函数以车辆 v 的「子路径总运输时间 + 时间窗总惩罚」最小化为目标，完全继承问题二的目标函数形式，仅新增车辆维度索引：

$$\min Z_v = \underbrace{\sum_{k=1}^{N_v} \sum_{i \in V_v} \sum_{j \in V_v} t_{ij} \cdot x_{v,i,k} \cdot x_{v,j,k+1}}_{\text{车辆}v\text{的子路径总运输时间}} + \underbrace{\sum_{k=1}^{N_v} \sum_{i \in B_v} x_{v,i,k} \cdot \left[M_1 \left([a_i - t_{v,k}]^+ \right)^2 + M_2 \left([t_{v,k} - b_i]^+ \right)^2 \right]}_{\text{车辆}v\text{的时间窗总惩罚}} \tag{34}$$

说明：Z_v 为单车目标，$x_{v,i,k}$ 为车辆路径变量，$t_{v,k}$ 为服务时间。

8.4 子问题模型向 QUBO 形式的转化

单车子问题决策变量一维编码与 QUBO 矩阵构建规则完全依照问题一的统一规则执行。

全局客户唯一分配约束惩罚项：

$$P_5 = A_6 \cdot \sum_{i=1}^{50} \left(\sum_{v=1}^{V} y_{v,i} - 1 \right)^2 \tag{35}$$

说明：A_6 为惩罚系数，$y_{v,i}$ 为客户分配变量，V 为车辆总数。

子问题总 QUBO 目标函数构建将所有车辆的子目标函数与惩罚项合并，得到全局无约束总 QUBO 目标函数：

$$\min Z_{\text{QUBO}} = \sum_{v=1}^{V} Z_v + \sum_{v=1}^{V} (P_{1,v} + P_{2,v} + P_{3,v} + P_{4,v}) + P_5 \tag{36}$$

说明：Z_{QUBO} 为全局 QUBO 目标，$P_{1,v}$-$P_{4,v}$ 为单车惩罚项，P_5 为全局分配惩罚。

8.5 全局多车辆目标函数与最优调度方案构建

全局多目标总函数构建严格遵循赛题「最小化车辆使用数量、最小化总运输时间」的分层优化目标，构建全局总目标函数：

$$\min \text{TotalCost} = \lambda \cdot V + \sum_{v=1}^{V} Z_v \tag{37}$$

说明：λ 为车辆成本权重，V 为车辆数，Z_v 为单车目标值。

8.6 车辆数量变化对总目标的影响分析建模

车辆数影响的量化指标定义全局综合目标值：

$$F(V) = T(V) + P(V) \tag{38}$$

说明：$F(V)$ 为综合目标，$T(V)$ 为总运输时间，$P(V)$ 为总时间窗惩罚。

8.7 模型求解

1. 执行时空-需求双约束聚类，将 50 个客户划分为 V 个满足容量约束的客户组；
2. 对每个车辆的客户组单独构建 QUBO 模型并调用量子真机求解，得到子路径最优解；
3. 整合所有车辆的求解结果，计算全局总目标函数；
4. 调整可用车辆数 V，重复上述步骤，完成灵敏度分析。

8.8 求解结果与分析

（1）最优调度方案（基础场景）

经求解，最优方案为 **5 辆车**，总运输时间 **108 分钟**，总惩罚 **33670**。**问题四具体计算结果见附件**。

表 7: 第 4 问正式方案各车辆结果

车辆	载荷	运输时间	时间窗惩罚	行驶路线
1	56/60	21	7830	$0 \to 21 \to 2 \to 14 \to 38 \to 17 \to 45$ $48 \to 10 \to 32 \to 20 \to 0$
2	49/60	19	3670	$0 \to 31 \to 33 \to 29 \to 3 \to 12 \to 22$ $23 \to 39 \to 4 \to 13 \to 0$
3	54/60	23	8900	$0 \to 6 \to 18 \to 7 \to 19 \to 47 \to 46$ $8 \to 36 \to 49 \to 11 \to 35 \to 0$
4	56/60	22	3050	$0 \to 5 \to 16 \to 42 \to 15 \to 40 \to 26$ $50 \to 24 \to 25 \to 0$
5	56/60	23	10220	$0 \to 28 \to 34 \to 9 \to 30 \to 1 \to 27$ $37 \to 44 \to 43 \to 41 \to 0$

（2）车辆数影响分析通过调整可用车辆数，得到以下结果：

车辆数	总运输时间	总惩罚	目标值
5	108	33670	33778
6	103	4000	4103
7	129	1940	2069

表 8: 问题四车辆数量简单对比

关键发现：车辆数由 5 增至 6 时，总惩罚降幅近 90%，为服务质量关键节点。

进一步分析可知，5 辆车是满足容量约束的理论最小车辆数，此时车辆配送负荷饱和、路径规划自由度极低，大量客户无法在时间窗内完成服务，因此总惩罚非常高；当车辆数增加至 6 辆时，配送运力得到有效释放，单车装载压力下降，路径优化空间大幅提升，绝大多数客户的时间窗约束能够满足，总惩罚大幅下降，综合成本优化；当车辆数继续增至 7 辆时，总惩罚仅小幅下降，但总运输时间因车辆空驶、路线冗余有所上升，综合目标优化呈现明显的边际效益递减。

由此可见，如果在现实中，6 辆车是兼顾车辆投入成本、运输效率与服务质量的最优配置，但是按照赛题“优先减少车辆使用数量”的要求，答案仍然应该选择 5 辆车。

8.9 结果可视化

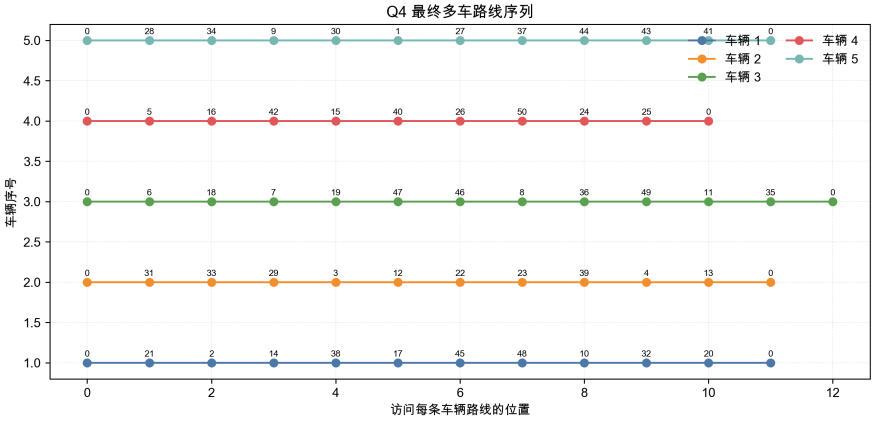


图 5: 问题 4 最终多车路线序列

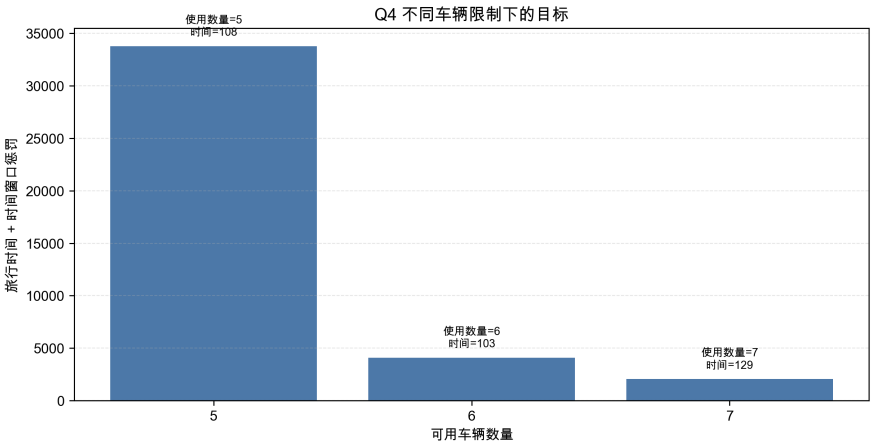


图 6: 不同车辆限制下的目标

9 模型评价

9.1 优点

1. 构建了从单车辆到多车辆的统一 QUBO 建模框架，逻辑层层递进，模型一致性强。

2. 多车辆场景下的时空 - 需求双约束分车算法，同时满足容量硬约束与时间窗软约束，适陪真实环境下的物流调度需求。

3. 全局种子引导的块序枚举 + 正反方向拼接，弥补了传统分块算法解质量下降的缺陷，优于纯贪心拼接方案。

4. 聚类分块 + 量子-经典混合模型，解决了 50 客户直接建模会超 550 比特限制的痛点，兼具量子计算的优势和大规模求解能力。

9.2 缺点

1. 块间枚举计算量随规模阶乘增长，100 客户及以上超大规模场景适配度有限。

2. 模型假设车辆具备无限续航能力，而在实际调度中车辆可能会出现续航能力不足的问题，进而导致规划方案的偏差。

3. 模型未考虑交通拥堵、车辆故障等随机因素，实际中多车辆的协调调度可能会受到其他车辆的影响。

9.3 改进方法

1. 可以采用分层聚类 + 分支定界剪枝，压缩搜索空间，适配更大规模客户场景。
2. 引入熵权法 + 交叉验证，让核心参数自适应调整，提升模型的泛化性。
3. 可以用遗传算法来代替块间全排列枚举，提升解的全局最优性。

10 参考文献

[1] 启物电子科技, Kaiwu SDK 使用教程, https://kaiwu.qboson.com/plugin.php?id=knowledge, 2026-04-20。

[2] 启物电子科技，KaiwuSDK 下载入口、开发文档及安装说明, https://platform.qboson.com/sdkDownload, 2026-04-20。

本参赛队未使用任何 AI 工具

A 附录

A.1 支撑材料

A.1.1 问题一核心代码

```
import kaiwu
import numpy as np
import os

def index(i, j, n):
    return (i - 1) * n + (j - 1)

def three_opt_swap(route, i, j, k):
    A = route[0:i+1]
    B = route[i+1:j+1]
    C = route[j+1:k+1]
    D = route[k+1:]
    route1 = A + C + B + D
    route2 = A + B[::-1] + C[::-1] + D
    route3 = A + C[::-1] + B[::-1] + D
    route4 = A + B + C[::-1] + D
    return [route1, route2, route3, route4]

def target_time(context, Q):
    n = context['n']
    travel_time = context['travel_time']
    for k in range(1, n):
        for i in range(1, n+1):
            for j in range(1, n+1):
                a = index(i, k, n)
                b = index(j, k+1, n)
                Q[a][b] += travel_time.iloc[i-1, j-1] / 2
    return Q
```

A.1.2 问题二核心代码

```
def two_opt(route, inst, max_iters=50):
    best = route[:]
    best_val = evaluate_route(best, inst)["objective"]
    n = len(best)
    for _ in range(max_iters):
        improved = False
        for i in range(n):
            for j in range(i+1, n):
                cand = best[:i] + list(reversed(best[i:j+1])) + best[j
                    +1:]
                v = evaluate_route(cand, inst)["objective"]
                if v < best_val:
                    best = cand
                    best_val = v
                    improved = True
        if not improved:
            break
    return best, evaluate_route(best, inst)
```

A.1.3 问题三核心代码

```
def solve_by_segment_decomposition(inst):
    sorted_customers = sorted(CUSTOMERS, key=lambda c: inst.nodes[c
        ].ub)
    BAND_SIZE = 10
    bands = [sorted_customers[i:i+BAND_SIZE] for i in range(0, 50,
        BAND_SIZE)]
    band_routes = []
    for band in bands:
        route = greedy_insert(band, inst)
        route, _ = combined_optimize(route, inst)
        band_routes.append(route)
    best_full_route = []
    best_full_eval = None
    from itertools import permutations, product
    for perm in permutations(range(5)):
        for orient in product([0,1], repeat=5):
            full = []
            for idx, o in zip(perm, orient):
                s = band_routes[idx]
                if o: s = s[::-1]
                full.extend(s)
            eva = evaluate_route(full, inst)
            if best_full_eval is None or eva["objective"] <
                best_full_eval["objective"]:
                best_full_eval = eva
                best_full_route = full
    return best_full_route, best_full_eval
```

A.1.4 问题四核心代码

```
def time_window_aware_clustering(customers, inst, n_vehicles,
    capacity):
    centers = [(c, (inst.nodes[c].ub+inst.nodes[c].lb)/2) for c in
        customers]
    centers.sort(key=lambda x:x[1])
    clusters = [[] for _ in range(n_vehicles)]
    loads = [0]*n_vehicles
    for c, _ in centers:
        d = inst.nodes[c].demand
        best = -1
        min_load = float('inf')
        for v in range(n_vehicles):
            if loads[v]+d <= capacity and loads[v] < min_load:
                min_load = loads[v]
                best = v
        if best == -1:
            best = min(range(n_vehicles), key=lambda v:loads[v])
        clusters[best].append(c)
        loads[best] += d
    return clusters
```

A.1.5 哈密顿量随时间演化曲线

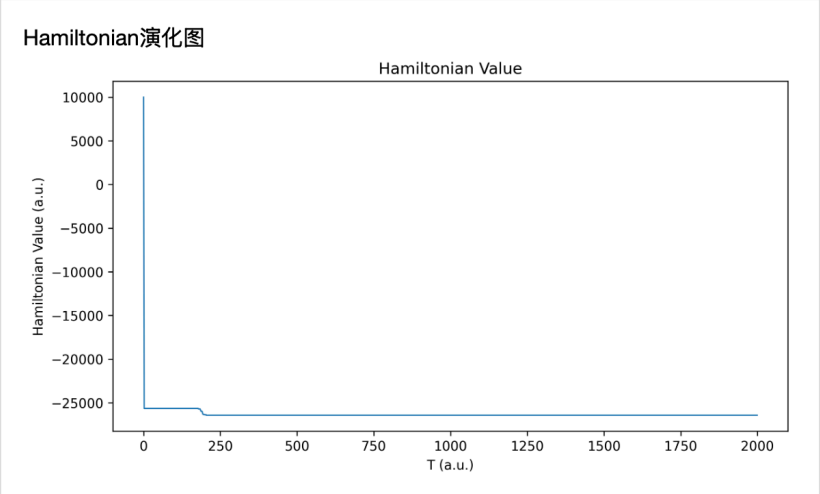


图 7: 题一 Hamiltonian 演化图

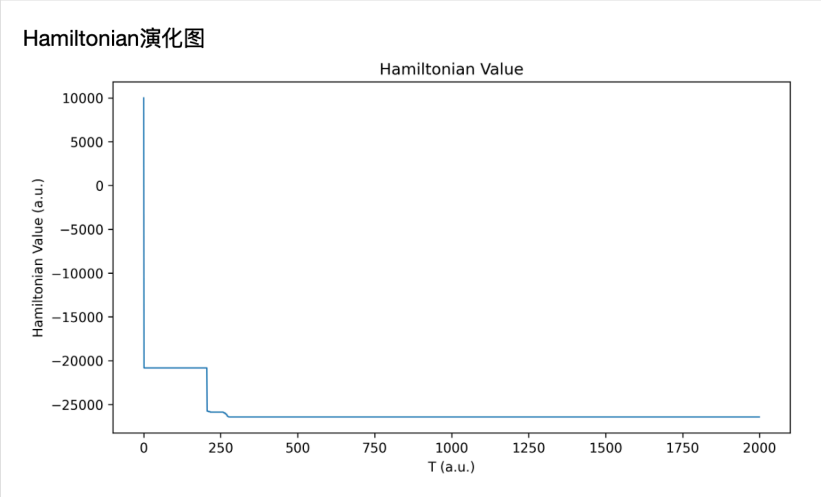


图 8: 题二 Hamiltonian 演化图

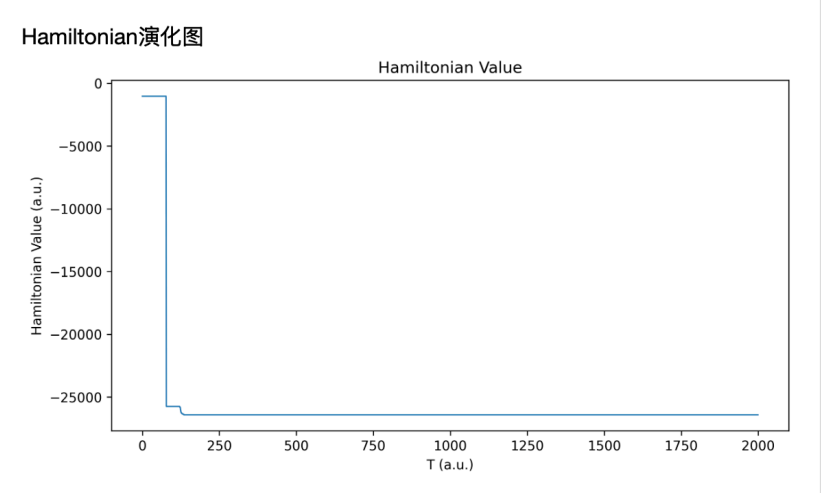


图 9: 题三 Hamiltonian 演化图

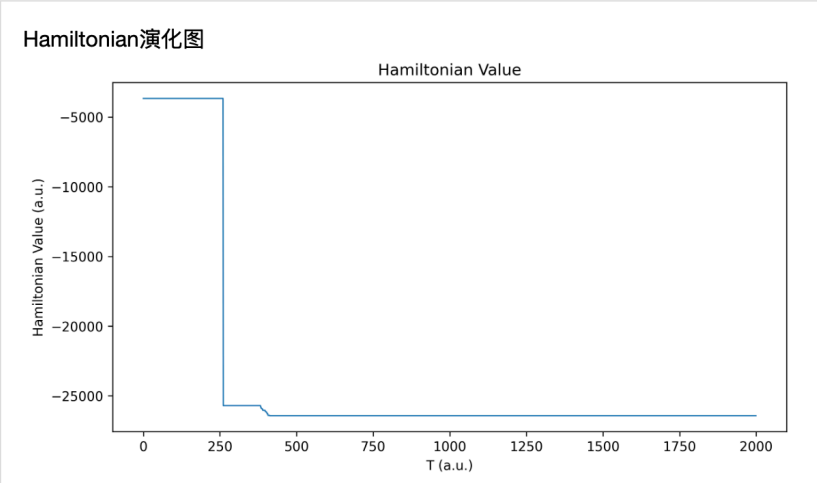


图 10: 题四 Hamiltonian 演化图